

day 14

📌 01. MongoDB Data Modeling (Relationships)

mongo day 3

💡 المفهوم الأساسي عند تصميم قواعد بيانات Document-Based في MongoDB، يتوفر نمطان رئيسيان لبناء العلاقات بين الـ Documents: **Embedding** و **Referencing**.

1. Embedded Data Model (Denormalization)

تضمين البيانات أو الـ Arrays داخل الـ Document نفسه بشكل مباشر.

- **المميزات:** أداء عالٍ وسريع جداً في القراءة (Read Performance) لأنك تجلب البيانات كلها بـ Query واحدة بدون الحاجة لـ Joins.
- **العيوب:** تكرار البيانات (Data Duplication)، واحتمالية تضخم حجم الـ Document لتتخطى الحد الأقصى للمونجو وهو 16MB.
- **الاستخدام الأنسب:** علاقات 1-1 أو 1-Few (مثل: مستخدم وعناوينه المحفوظة).

JSON

```
// Example: Embedded Department inside Employee
{
  "_id": 1,
  "name": "Eman",
  "department": {
    "name": "SD",
    "phone": "01234"
  }
}
```

2. Referenced Data Model (Normalization)

فصل البيانات في Collections مختلفة والربط بينها باستخدام المرجعية (IDs) بنفس فكرة الـ Foreign Key في SQL.

- **المميزات:** يمنع تكرار البيانات، مرونة أعلى عند تحديث البيانات المركزية (زي تعديل اسم الكلية أو القسم).
- **العيوب:** يستدعي عمليات ربط (زي \$lookup) مما يقلل من السرعة مقارنة بالـ Embedding.
- **الاستخدام الأنسب:** علاقات 1-Many أو Many-to-Many (مثل: الطلاب والكورسات).

JSON

```
// Employee Document
{ "_id": 1, "name": "Eman", "department_id": 3 }

// Department Document
{ "_id": 3, "name": "SD", "phone": "01234" }
```

📌 02. Aggregation Pipeline & Operators

تعتمد على **MongoDB** هي إطار عمل مقتدر لمعالجة وتحليل البيانات بداخل **Aggregation Pipeline** **i** **!** حيث تدخل المخرجات الخاصة بكل مرحلة ك مدخلات للمرحلة التالية، **(Pipelines)** فكرة المراحل المتتالية

الصيغة العامة: `db.collection.aggregate([ Stage1, Stage2, ..., StageN ])`

أشهر الـ Aggregation Stages 🛠️

Stage Operator	الوظيفة والشرح
\$match	يفلتر المستندات بناءً على شرط معين (مثل WHERE في SQL).
\$project	يعيد تشكيل المستند (إظهار/إخفاء حقول، أو إنشاء حقول حسابية جديدة بـ concat\$).
\$group	يجمع البيانات بناءً على مفتاح محدد (id_) ويطبق حسابات تراكمية (Accumulators).
\$sort	يعيد ترتيب المستندات تصاعدياً (1) أو تنازلياً (-1).
\$limit / \$count	يحدد عدد المستندات الناتجة أو يحسب إجمالي عددها.
\$out	يكتب نتائج الـ Aggregation مباشرة في Collection جديدة في الداتابيز.
\$lookup	ينفذ عملية Left Outer Join لربط البيانات بين كوليكشن وأخرى.

الـ Accumulators الخاصة بـ \$group 📊

تستخدم داخل مرحلة الـ \$group لحساب قيم إحصائية:

- **\$sum**: يحسب المجموع.
- **\$avg**: يحسب المتوسط الحسابي.
- **\$min / \$max**: يحسب أقل وأعلى قيمة.

📌 03. MongoDB Indexing

⚠️ لماذا نستخدم الـ **Indexing**؟ بدون الفهارس (Indexes)، يضطر سيرفر **MongoDB** إلى عمل **COLLSCAN (Collection Scan)** وقراءة كل المستندات في الداتابيز للبحث عن النتيجة.

عند إضافة Index، يستخدم المونجو B-Tree لتنفيذ IXSCAN (Index Scan) للقفز المباشر للبيانات المطلوبة، مما يضاعف سرعة الـ \$match, find, و \$sort.

? أسئلة إنترفيو متوقعة (Interview Q&A)

② **Q1: When should you prefer Embedded Data Models over Referenced Data Models in MongoDB?** عندما تكون العلاقة بين Embedded الإجابة: نفضل استخدام الـ Embedded وكانت البيانات المرتبطة يغلب عليها الاستعلام والقراءة معاً، ولا نخشى من Few-to-1 أو 1-to-1 البيانات بنظام 1 MB. إلى حد الـ 16 Document وصول حجم الـ

② **Q2: What is the difference between \$match stage in Aggregation and \$project أو \$group وتكرر المخرجات المفلترة للمراحل التالية مثل (Pipeline) أسطول معالجة البيانات التي ترجع النتيجة النهائية للمستخدم مباشرة \$find().** \$match تعمل كـ الإجابة: كلاهما يقوم بالفلتر بناءً على شروط، ولكن \$find() method? \$project أو \$group وتكرر المخرجات المفلترة للمراحل التالية مثل (Pipeline) أسطول معالجة البيانات التي ترجع النتيجة النهائية للمستخدم مباشرة \$find().

② **Q3: How does \$lookup perform a join in MongoDB?** Left Outer Join تقوم بعمل الإجابة: \$lookup Join بين Collection رئيسية و Collection أخرى عن طريق مطابقة الـ localField في الأولى بالـ foreignField داخل حقل جديد يحدد بـ Array في الثانية، وتضع قائمة النتائج المطابقة كـ as .